

Introduction to Features:

Computer Vision Fundamentals

Computer Vision Team

What Are Features?

- Features are distinctive elements or structures in images.
- Used for matching, recognition, and reconstruction.

Common Types:

Edges

Corners

Blobs

Ridges

Why Are Features Important?

| Enable Core Tasks

- Object recognition
- Object tracking
- Panorama stitching

| Provide Robustness

- Scale variations
- Rotation changes
- Illumination shifts

Types of Features

Edge Features

Detected by gradients,
e.g., Canny

Corner Features

Harris, Shi-Tomasi, FAST

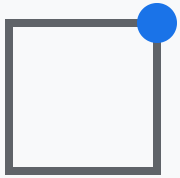
Blob Features

Difference of Gaussians,
SIFT, SURF

Descriptors

SIFT, ORB, BRIEF

Visualizing Features (Examples)



**Harris
Corners**



SIFT



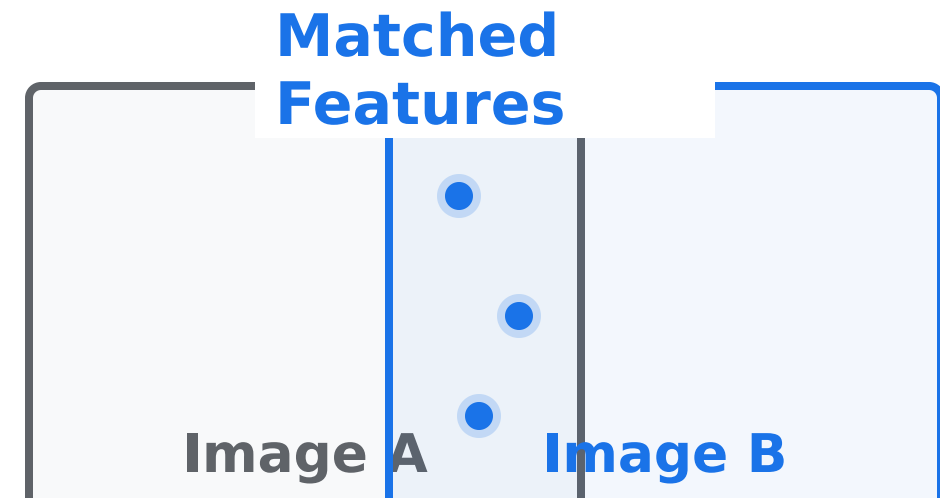
ORB

Variability: How different are the features from each other?

Repeatability: Can the same features be found in

Features in Panorama Images

- **The Goal:** Align overlapping images to create a seamless panorama.
- **The Challenge:** Finding common features between the overlapping regions.
 - **Feature Detection:** Identify distinct points (e.g., corners, edges) in each image.



How Panoramas Are Made

[01] Detect features

[02] Match features

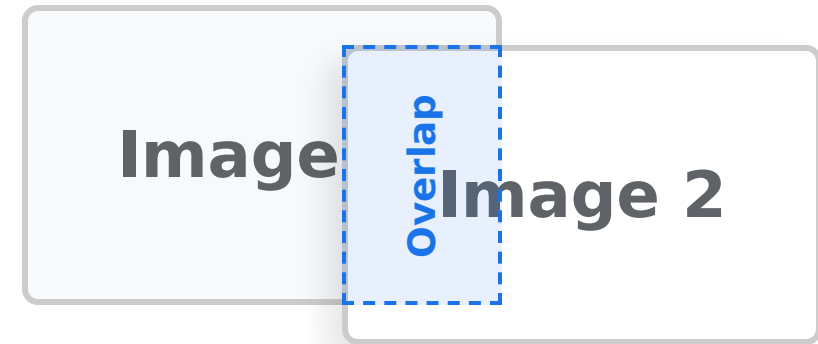
[03] Estimate transformation (homography)

[04] Blend

Panorama Example Image

- **Before State:** Capturing a scene requires taking sequential photos with a 20-30% overlap to ensure common features exist.
- **After State:** Computer vision algorithms detect features, align the images, and blend them into a final stitched panorama.

Before: Overlapping Photos



After: Stitched Panorama



Feature Detection Demo (Python / OpenCV)

- Python implementation of SIFT feature detection:

```
import cv2 as cv
import numpy as np

# Load the image in grayscale
img = cv.imread('target_image.jpg', cv.IMREAD_GRAYSCALE)

# Initialize the SIFT detector
sift = cv.SIFT_create()
```

Feature Detection Demo (Python / OpenCV)

```
# Detect keypoints and compute descriptors
keypoints, descriptors = sift.detectAndCompute(img, None)
# Draw keypoints on the original image
img_with_keypoints = cv.drawKeypoints(
    img,
    keypoints,
    None,
    flags=cv.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
```

Feature Detection Demo (Python / OpenCV)

- Python implementation of SIFT feature detection:

```
# Display the result
cv.imshow('SIFT Features', img_with_keypoints)
cv.waitKey(0)
cv.destroyAllWindows()
```

Feature Matching Example

Implementing Brute-Force Matching with Lowe's Ratio Test

```
# Match descriptors.
matches = bf.knnMatch(des1, des2, k=2)
# Apply ratio test
good = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good.append([m])
```

Feature Matching Example

```
# create BFMatcher object
```

```
bf = cv2.BFMatcher()
```

```
# cv2.drawMatchesKnn expects list of lists as matches.
```

```
img3 = cv2.drawMatchesKnn(img1, kp1, img2, kp2, good, None, flags)
```

Homography Estimation using OpenCV

Calculate the perspective transformation matrix between corresponding point sets:

```
import cv2

# Estimate the homography matrix between two sets of points
H, mask = cv2.findHomography(
    srcPoints=src_pts,
    dstPoints=dst_pts,
    method=cv2.RANSAC,
```

Panorama Stitching - Blending Example

- Applying the homography matrix to warp and blend images:

```
import cv2
import numpy as np

# Assuming img1, img2 are loaded and H is the homography matrix
# Get dimensions of the destination image
height, width = img1.shape[:2]

# Warp img2 to img1's perspective using cv2.warpPerspective
warped_img = cv2.warpPerspective(img2, H, (width + img2.shape[1], height))

# Blend the images (simple overlay for demonstration)
warped_img[0:height, 0:width] = img1
```

Common Feature Detectors in OpenCV

SIFT

Scale-Invariant Feature Transform

- **High Accuracy:**
Excellent for complex matching tasks.

ORB

Oriented FAST and Rotated BRIEF

- **Moderate Accuracy:**
Good for general use cases.

AKAZE

Accelerated-KAZE

- **Good Accuracy:**
Better than ORB, slightly below SIFT.

Summary: Computer Vision Features

- **Importance of Features**

Enable robust object recognition, tracking, and panorama stitching across varying scales and illumination.

- **Types of Features**

Categorized primarily into Edges (gradients), Corners (Harris, FAST), and Blobs (SIFT, SURF).

- **OpenCV Tools for Detection & Stitching**

Questions & Answers

Thank you for your attention.